

09 — Données & méthodologie

Table des matieres

[8.1 — Acquisition & normalisation des données](#)

[8.2 — Validation structurelle](#)

[8.3 — Cross-validation entre sources](#)

[8.4 — Reproductibilité & configuration retargetable](#)

[Réponses aux exercices](#)

Niveau : base → intermédiaire. **Prérequis** : aucun formel (utile : 01_fondations.md pour $p = 6/49$). Se lit bien à tout moment du parcours. **Couvre les fiches source** : 8.1 à 8.4 de concepts_FR.md. **Source des chiffres** : reports/phase0_validation_FR.md (figure fig01).

Idée directrice. Toute la rigueur des Phases 1 et 2 s’effondrerait sur des données douteuses : un historique troué fausserait les écarts, les séries, l’autocorrélation. Ce document décrit comment le projet **fonde sa confiance** : acquisition propre depuis la source officielle, validation structurelle, cross-validation entre sources (qui a même démasqué un bug), et un pipeline **reproductible et retargetable**. C’est la fondation vérifiée sur laquelle repose le verdict « indistinguishable du hasard ».

8.1 — Acquisition & normalisation des données

En une phrase

On extrait les tirages depuis la source **officielle Loto-Québec**, on les parse et on les met en forme (date, n1..n6, bonus, numéros triés par ligne) — 4 377 tirages de 1982 à 2025.

Intuition

Avant d’analyser, il faut des données propres et fiables. Le projet est d’abord parti d’un jeu public (Kaggle) avant de basculer vers la source officielle, plus complète et exacte. L’acquisition n’est pas un détail : les pièges de parsing (dates, HTML hétérogène) peuvent corrompre silencieusement le jeu.

Formalisation

Le pipeline d’acquisition (loader.py) produit une table normalisée : une ligne par tirage, colonnes date, n1..n6, bonus, **numéros triés par ligne** (un tirage est un *ensemble*, pas une séquence ordonnée). Pièges traités lors de l’extraction officielle (année par année) :

- **Virgule dans la date.** « June 12, 1982 » contient une virgule qui, non protégée, casse le découpage CSV → lire des colonnes explicites puis reconstruire la date.
- **Format des mois.** Mois en toutes lettres → directive %B (et non %m).
- **Tirage « Boule d'or » écarté.** Depuis 2022, un tirage annexe (code type 09377227-01) ne relève pas du 6/49 ; on ne garde que le tirage classique (6 numéros + complémentaire).
- **HTML hétérogène.** Le balisage varie selon les années (classe numerosGagnants fautive jusqu'à ~2023 puis numerosGagnants, guillemets variables, malformée en 2025) → parseur rendu **tolérant**.

Dans iAlexMG

- src/acquisition/loader.py : chargement + normalisation ; **4 377 tirages** du **12 juin 1982** au **31 décembre 2025**.
- Le **tri par ligne** garantit qu'un tirage est traité comme un ensemble (cohérent avec le sans-remise, §0.3).

Pièges & malentendus

- **Faire confiance à un jeu « complet » sans vérifier.** Le jeu Kaggle paraissait complet mais était lacunaire (§8.3).
- **Traiter un tirage comme ordonné.** Les positions n'ont pas de sens : on trie (§0.2, symétrie des positions).
- **Parsing de dates naïf.** Virgules et noms de mois sont des pièges classiques.

Pour vérifier ta compréhension

1. Pourquoi trier les numéros d'un tirage par ligne ?
2. Quel piège la virgule de « June 12, 1982 » pose-t-elle en CSV ?
3. Pourquoi écarter le tirage « Boule d'or » ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Robustesse de parsing** : tolérer les variations de balisage plutôt que coder en dur.
- **Idempotence** : un pipeline rejouable produit toujours le même jeu propre.



8.2 — Validation structurelle

En une phrase

On vérifie automatiquement que chaque tirage respecte les invariants du jeu : bornes [1,49], 6 numéros distincts, dates uniques, complétude temporelle.

Intuition

Une fois les données chargées, on ne les croit pas sur parole : on teste des **invariants** que tout tirage légitime doit satisfaire. Le moindre écart signalerait une corruption.

Formalisation

Contrôles structurels (validate.py) sur les 4 377 tirages — **tous conformes** :

Contrôle	Résultat
Nombre de tirages	4 377
Numéros dans [1,49] (6 principaux + boni)	conforme
6 numéros principaux distincts par tirage	conforme
Dates dupliquées	0
Plage temporelle	1982-06-12 → 2025-12-31

Complétude temporelle (figure 1a) : montée en cadence initiale (1982-1985), puis **stabilisation à ~104 tirages/an dès 1986**, sans aucun creux jusqu'en 2025. Conséquence : les analyses temporelles ne sont **pas faussées** par des trous (aucune restriction de période nécessaire).

🔗 **Mythe corrigé.** Le passage au bi-hebdomadaire est souvent daté de 1995 ; les données officielles le situent **vers 1986**. Le jeu Kaggle suggérait 1989 — un artefact de ses tirages manquants.

Dans iAlexMG

- src/acquisition/validate.py ; **figure 1** (couverture des 49 boules, complétude annuelle).
- La validation est la **porte d'entrée** des Phases 1 et 2 : pas d'analyse sur des données non validées.

Pièges & malentendus

- **Supposer la complétude sans la mesurer.** Un creux d'historique fausserait gaps, séries, ACF.

- **Confondre absence d'erreur de format et exactitude.** La validation structurelle vérifie la cohérence interne ; l'exactitude vient en plus de la cross-validation (§8.3).

Pour vérifier ta compréhension

1. Cite trois invariants vérifiés par la validation structurelle.
2. Pourquoi la complétude temporelle est-elle cruciale pour les tests temporels ?
3. Depuis quand l'historique est-il sans trou, et pourquoi est-ce important ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Invariants de jeu** : encoder les règles comme tests automatiques.
 - **Détection de trous** : dénombrement par année comme signal d'alerte.
-

8.3 — Cross-validation entre sources

En une phrase

On recoupe trois sources (officielle, Kaggle, relevé manuel) pour adjuger la source fiable et démasquer les erreurs — dont un bug de décalage sur 1998-1999.

Intuition

Une donnée n'est digne de confiance que corroborée. En comparant plusieurs sources, on valide les accords et on enquête sur les désaccords. Mieux : une source qui **se contredit elle-même** est fautive, sans même recourir à une référence externe.

Formalisation

- **Officiel vs Kaggle** : sur **3 621 dates communes**, **3 612 identiques** ($\approx 99,8\%$) ; les **9 écarts** sont des erreurs de Kaggle (l'officiel fait foi).
- **Un bug attrapé par cohérence interne.** Le relevé manuel présentait **214 discordances** avec Kaggle, **concentrées sur 1998-1999**. Un test de cohérence interne révèle le coupable : l'onglet 1998 partageait **104 tirages sur 105** avec l'onglet 1997. Or un tirage est **unique** (1 chance sur $\binom{49}{6} \approx 14$ millions) : 104 répétitions sont **structurellement impossibles** par hasard. Côté officiel et Kaggle : **0 doublon interne**. Conclusion : les onglets 1998-1999 du relevé manuel étaient **décalés d'un an**.

 **Leçon.** Face à des sources qui se contredisent, chercher d’abord une **incohérence interne** : une source qui se contredit elle-même est fautive.

Dans iAlexMG

- Choix de la **source canonique** (officielle Loto-Québec) en Phase 0 ; Kaggle conservé uniquement pour la cross-validation.
- Données finales : **4 377 tirages**, complets, cohérents et corroborés.

Pièges & malentendus

- **Trancher entre sources « à l’aveugle »**. D’abord chercher une incohérence interne objective.
- **Croire « public = fiable »**. Le jeu Kaggle, pourtant populaire, était lacunaire et partiellement erroné.

Pour vérifier ta compréhension

1. Quel taux de concordance officiel/Kaggle observe-t-on, et qui fait foi en cas d’écart ?
2. Comment a-t-on démasqué le bug 1998-1999 *sans* référence externe ?
3. Pourquoi 104 tirages identiques entre deux années sont-ils « impossibles » ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Cohérence interne vs externe** : deux leviers de validation complémentaires.
- **Record linkage** : appairer des enregistrements de sources hétérogènes.

8.4 — Reproductibilité & configuration retargetable

En une phrase

Le pipeline est en scripts, avec seeds fixés et données régénérables ; le jeu est paramétré par un seul objet gelé (GameConfig), de sorte qu’on peut le re-cibler en changeant un champ.

Intuition

Une analyse scientifique doit être **rejouable à l’identique** par un tiers. Et un bon design isole les paramètres du jeu (49 boules, 6 tirées) en un seul endroit, pour que tout le code s’adapte automatiquement à un autre loto.

Formalisation

- **Déterminisme** : tous les générateurs pseudo-aléatoires (Monte-Carlo §2.8, permutation §2.9, modèles §5) sont **seedés** ; les figures et p-values sont reproductibles.
- **Données régénérables** : la source brute est versionnée ; les données traitées (.parquet / .xlsx) sont **gitignorées** mais reconstructibles par le pipeline.
- **Configuration retargetable** : un objet GameConfig **gelé** (@dataclass(frozen=True)) porte $n_balls = 49$, $n_drawn = 6$, etc. La constante pivot $single_draw_probability = n_drawn / n_balls = 6/49$ est **calculée**, pas codée en dur (§0.2). Changer de jeu (p. ex. 7/50) = changer un champ, et tout le pipeline suit.
- **Gel des dépendances** : environment.yml (env conda iaexmg_649) fige les versions.

Dans iAlexMG

- config.py (GameConfig), seeds (numpy default_rng), environment.yml.
- Garantit que le verdict est **vérifiable** et que l'étude est **transposable** à un autre jeu de loterie sans réécrire le code.

Pièges & malentendus

- **Coder 6/49 en dur**. Casse la retargetabilité ; calculer depuis la config.
- **Oublier les seeds**. Résultats non reproductibles (Monte-Carlo, permutation, init des réseaux).
- **Versionner les données lourdes traitées**. Préférer les régénérer ; versionner la source brute.

Pour vérifier ta compréhension

1. Pourquoi $single_draw_probability$ est-elle *calculée* et non codée en dur ?
2. Quel rôle jouent les seeds dans la reproductibilité ?
3. Pourquoi geler la configuration (frozen=True) ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Environnements gelés** (environment.yml, lockfiles) : reproductibilité des dépendances.
 - **Configuration centralisée** : un seul point de vérité pour les paramètres du jeu.
-

Réponses aux exercices

8.1

1. Parce qu'un tirage est un **ensemble** non ordonné : les positions n'ont pas de sens (symétrie, §0.2) ; trier garantit une représentation canonique.
2. Non protégée par des guillemets, la virgule est interprétée comme un **séparateur de colonnes** → date découpée, colonnes décalées.
3. Parce que le tirage « Boule d'or » (depuis 2022) **ne relève pas du 6/49** ; l'inclure polluerait l'historique.

8.2

1. Bornes [1,49], 6 numéros **distincts** par tirage, dates **uniques** (et complétude temporelle).
2. Parce qu'un trou créerait de faux écarts/séries et fausserait l'autocorrélation ; la complétude évite cet artefact.
3. Depuis **~1986** (≈ 104 tirages/an, sans creux) ; cela permet d'analyser tout l'historique sans restreindre la période.

8.3

1. **$\approx 99,8 \%$** ($3\,612 / 3\,621$) ; en cas d'écart, la **source officielle** fait foi.
2. Par **cohérence interne** : l'onglet 1998 du relevé manuel dupliquait 104/105 tirages de 1997 — impossible pour des tirages uniques → la source se contredisait elle-même.
3. Parce qu'un tirage a 1 chance sur $\binom{49}{6} \approx 14$ millions ; 104 répétitions à l'identique sont structurellement impossibles par hasard.

8.4

1. Pour rester **retargetable** : en la calculant depuis `n_drawn/n_balls`, changer de jeu ne demande que de modifier la config, et tout le pipeline suit.
2. Ils rendent les tirages pseudo-aléatoires (Monte-Carlo, permutation, init des modèles) **rejouables à l'identique**.
3. Pour empêcher toute modification accidentelle des paramètres du jeu en cours d'exécution (un seul point de vérité, immuable). ``